Array

In Toth le array non possiedono un tipo specifico e ogni elemento può essere di tipo differente, ecco un esempio:

```
var arr = [“Hello”, 99, 3.0];
```

Come ottenere un elemento da un array:

```
arr[0] \ottiene l’elemento nella posizione 0 in questo caso: Hello\
```

Per modificare un elemento di una Array bisogna affidarsi a una funzione

```
var arr = arrEdit(arr, 0, “Bye”); \adesso l’array è [“bye”, 99, 3.0]\
```

Ecco una lista delle funzioni di tipo array:

```
push(arr,"gg")
```
Aggiunge un elemento alla fine di una array

```
pushFirst(arr,"gg")
```
Aggiunge un elemento all’inizio di una array

```
pushAt(arr,"gg",1)
```
Aggiunge un elemento nella posizione specificata in una array

```
pop(arr,1)
```
Rimuove il l’elemento nella posizione specificata

```
shift(arr)
```
Rimuove il primo elemento di una array

```
len(arr)
```
Restituisce il numero di elementi di una array

```
first(arr)
```
Restituisce il primo elemento di una array

```
last(arr)
```
Restituisce l’ultimo elemento di una array

```
reverse(arr)
```
inverte la posizione di tutti gli elementi di una array

```
indexOf(arr, “gg”)
```
Cerca nell’array e restituisce la posizione di un elemento

```

```

Cicli

In Toth sono presenti due tipi di cicli **For** e **While**

Il For in Toth ha bisogno di una definizione inziale del contatore, una condizione, ed infine il numero di **step** in cui il contare si sposterà ad ogni iterazione.

```
for(var i =0, i < 5, 1){
          println(i);
}
\*
Output:
0
1
2
3
4
*\
```

In questo esempio il ciclo For esegue la funzione **println** ad ogni iterazione ripetendolo per 5 volte

Se al posto dell'addizione si vuole usare un'altra operazione negli step è possibile specificando il contatore:

```
for(var i =0, i != 5, -1)...
for(var i =0, i != 5, i * 2)...
for(var i =0, i != 5, i / 2)...
```

In Toth nel While basta definire la condizione e il loop al suo compimento si attiverà.

```
while(true){
    println("gg");
}
\*
Output:
gg
gg
gg
...
*\
```

In questo esempio il While esegue all'infinito la funzione **println**

Convertire tipi

In Toth è possibile convertire tipi in modo implicito o esplicito

Per implicito si intende una conversione più diretta come questa:

```
var str = ”Anni: ” + 22; \Qui il 22 viene convertito in stringa e risulta: “Anni: 22”\
var int = 10 + 1.0; \Qui il 1.0 viene convertito da float ad integer risultando: 11\
```

Mentre per esplicita si intende una conversione che fa uso di funzioni.

Esempi di funzioni per la conversione:

```
var int = toInt(“10”);
var str = toString(5);
var str = toString([“foo”, “bar”]);
```

Se la funzione non riesce a convertire correttamente un valore ritorna valore null

Definire una funzione

In Toth anche le funzioni sono riconosciute come variabili pertanto bisogna definirle con il prefisso **var** e come valore utilizzare **fn** seguito dai parametri che si vogliono utilizzare nella funzione

```
var foo = fn(x1, x2, x3){}
```

Ecco come definire una semplice funzione che ritorna l'input incrementato di 2:

```
var plusTwo = fn(x){
          return x + 2;
}
```

Inoltre in Toth è possibile anche non specificare il **return** semplicemente dichiarando un espressione fuori funzione:

```
var plusTwo = fn(x){
          x + 2;
}
```

Come richiamare una funzione:

```
plusTwo(2) \4\
```

la funzione ritornerà **4**

Definire Variabili

Come definire una variabile:

var NOME = VALORE

Il nome può essere composto da lettere, numeri e '_' ma deve sempre iniziare con una lettera

Ecco alcuni esempi di variabili con nomi validi in Toth:

```
var foo = 0;
var Foo = 0;
var foo_ = 0;
var foo10 = 0;
var FoO_10 = 0;
```

Tipi in Toth:

```
var string = "Hello World!";

var integer = 100;

var float = 100.0;

var array = ["elem1", "elem2"];

var hash = {"age":18, "name":"Gionny"};

var function fn(x){return x;}
```

In Toth è totalmente consentito sovrascrivere o ridefinire una variabile.

Per ottenere il tipo di una variabile si può utilizzare la funzione **typeof**

```
typeof(100);
typeof("Hello World!");
```

Gestire mouse e tastiera

```
cursorPos()
```

```
keyClicked()
```

Hash

In Toth le Hash (Hashmap o Dictionary) non possiedono un tipo specifico per le chiavi e per i valori

```
var hash={"nome":"gionny","età":230,2.0:”due”};
```

per ottenere un valore da una **Hash** basta inserire la chiave tra due quadre:

```
hash[“nome”]
```

se la chiave non esiste ritornerà **Null**

ecco una lista delle funzioni disponibili per le Hash:

```
len(hash)
```

restituisce il numero di valori presenti nell’Hash

```
join(hash, “, ”)
```

unisce le chiavi e i valori in una stringa

```
hashEdit(hash,”età”,5)
```

modifica il valore indicato dalla chiave

```
pop(hash,”età”)
```

rimuove la chiave specificata

Importazione di moduli

In Toth è possibile “importare” moduli usando la funzione **using.**

L’importazione di essi però è solo un richiamo/esecuzione del file per questo motivo vengono introdotte tutte le variabili/funzioni con il nome dato nel modulo. Queste caratteristiche permettono di sovrascrivere funzioni e variabili fuori dal modulo. Inoltre è consentito richiamare più volte lo stesso module in ogni **Ambiente**.

Ecco un esempio:

```
using "module"
```

basta inserire il nome del modulo ignorando l’estensione “.th”

Input e output

Stampare valori in Toth:

```
print(“Valore”); \Stampa: Valore\
println(“Valore”); \Stampa in una nuova linea: Valore\
CLRprint(“Valore”, 255, 0, 0); \Stampa a colori utilizzando il codice RGB: Valore\
CLRprintln(“Valore”, 255, 0, 0); \Stampa in una nuova linea a colori utilizzando il codice RGB: Valore\
printPNG("C:\path\image.png"); \Ridimensiona a 32x32 e stampa un’immagine png\
```

le funzioni **print** e **println** accettano infinti valori e li stampano separati da un spazio

```
println(“Valore1”, “Valore2”); \Stampa: Valore1 Valore2\
```

Leggere valori

```
readln()
```

Pulire la console

```
clearConsole()
```

Manipolazione dei File

Come creare un file vuoto in Toth

```
createFile("file.txt")
```

Rimuovere un file

```
removeFile("file.txt")
```

Creare un file scritto (la funzione non aggiungerà testo al file ma lo sovrascriverà)

```
writeFile("file.txt","text")
```

Restituisce il contenuto di un file

```
readFile("file.txt")
```

Creare una cartella

```
createDir("cartella")
```

tutte le funzione elencate accettano solo percorsi assoluti per questo motivo è importante notare che se si vuole specificare la posizione di un file va specificato il percorso per intero ES:
readFile("desktop/project/file.txt")

Operatori

Operatori aritmetici

x + y Addizione

x - y Sottrazione

x * y Moltiplicazione

x / y Divisione

x % y Percentuale

x ^ y Potenza

Operatori di confronto

x == y Uguale

x != y Non uguale

x <= y Minore o uguale

x >= y Maggiore o uguale

x < y Minore

x > y Maggiore

Operatori logici

x || y Or

x && y And

Operatori di assegnazione

x = y Assegna

x += y Assegna addizione

x -= y Assegna sottrazione

x =: y Assegna globale

Random

In Toth per generare numeri casuali è possibile utilizzare la funzione:

```
randomInt(0, 10) \restituisce un numero casuale da 0 a 10\
```

Segnalare un errore

```
CLRprintln("ERROR: throwed error: test",200,0,0);
exit()
```

Equivale a stampare l'errore in rosso e uscire dal programma

```
return CLRprintln("ERROR: throwed error: test",200,0,0);
```

a differenza della versione precedente l'errore verrà stampato ma uscirà solo dall'ambiente corrente (come una funzione o un for).

```
throwError("test")
```

Questo è il metodo più completo per riportare un errore nel programma

String

Come definire una stringa in Toth:

```
var str = ”Hello World!”
```

Le stringhe accettano anche operatori come ‘\n’ (Newline)

```
var str = ”Hello World!\n”
```

è possibile usare alcune operazioni aritmetiche nelle stringhe:

```
var str = ”Hello” + ” World!” \Hello World!\

var str = ”Hello!” * 3 \Hello!Hello!Hello!\
```

ecco una lista delle funzioni per manipolare le stringhe:

```
len("stringa")
```

ritorna il numero di caratteri in una stringa

```
charAt("stringa",1)
```

ritorna il carattare di una stringa in una specifica posizione

```
replace("Hello World!","Hello","Goodbye")
```

sostituisce tutte le ricorrenze di una certa parola in una stringa con un’altra parola

```
substring("Hello World!",0,5)
```

estrae una porzione di stringa da una stringa

```
insert("HelloWorld!"," ",5)
```

inserisce una stringa allinterno di un’altra in una specifica posizione

```
split("ciao-23-fg","-")
```

divide una stringa usando un divisore di riferimento e ritorna una array

```
join(arr)
join(arr, ", ")
```

unisce una array concatenando gli elementi con una stringa(facoltativa) e ritorna stringa

```
toUpper("hello world!")
```
converte la stringa in maiuscolo

```
toLower("HELLO WORLD!")
```
converte la stringa in minuscolo

```
overplace("Hello WARld!","Wor",5)
```
sovrascrive una parte di stringa con la stringa immessa iniziando da una posizione specificata

```
bytelen("stringa")
```
ritorna il numero di byte in una stringa

```
contains("Hello World!","World")
```
restituisce **True** se la stringa contiene la porzione di stringa inserita

```
isNum("232")
```
restituisce **True** se la stringa è formata solo da caratteri numerici

```
isLetter("Hello World!")
```
restituisce **True** se la stringa è formata solo da caratteri letterali